

Files in Java

Java provides classes to create, read, write, and delete files.

Method	Purpose
<code>createNewFile()</code>	Creates a new file
<code>exists()</code>	Checks whether file exists
<code>getName()</code>	Returns file name
<code>getPath()</code>	Returns file path
<code>length()</code>	Returns file size
<code>delete()</code>	Deletes the file

Create a File

```
import java.io.File;
import java.io.IOException;

class Demo {
    public static void main(String[] args) {
        try {
            File f = new File("test.txt"); // f is the reference to that file.

            if (f.createNewFile()) {
                System.out.println("File created");
            } else {
                System.out.println("File already exists");
            }
        } catch (IOException e) {
            System.out.println("Error");
        }
    }
}
```

Check Whether File Exists

```
import java.io.File;

class Demo {
    public static void main(String[] args) {
        File f = new File("test.txt");

        if (f.exists()) {
            System.out.println("File exists");
        } else {
            System.out.println("File does not exist");
        }
    }
}
```

Delete a File

```
import java.io.File;

class Demo {
    public static void main(String[] args) {
        File f = new File("test.txt");

        if (f.delete()) {
            System.out.println("File deleted");
        } else {
            System.out.println("File not found");
        }
    }
}
```

Simple File Information Program

```
import java.io.File;

class Demo {
    public static void main(String[] args) {

        File f = new File("test.txt");

        System.out.println("Name: " + f.getName());
        System.out.println("Path: " + f.getPath());
        System.out.println("Exists: " + f.exists());
        System.out.println("Size: " + f.length() + " bytes");
    }
}
```

Character Stream vs. Byte Stream

Character Stream	Byte Stream
Works with characters	Works with bytes
FileReader	FileInputStream
FileWriter	FileOutputStream
Suitable for text	Suitable for images, audio, video
Characters	8-bit bytes

File Handling Using Character Stream

Write to a File

```
import java.io.FileWriter;
import java.io.IOException;

class Demo {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("test.txt");
```

```

        // it immediately erases/overwrites any existing content, even before you write
        a single character.
        fw.write("Hello Java");
        fw.write("\nWelcome to File Handling");

        fw.close();

        System.out.println("Data written");
    } catch (IOException e) {
        System.out.println("Error");
    }
}
}

```

Reading from a File

```

import java.io.File;
import java.util.Scanner;

class Demo {
    public static void main(String[] args) {
        try {
            File f = new File("test.txt");

            Scanner sc = new Scanner(f);

            while (sc.hasNextLine()) {
                String line = sc.nextLine();
                System.out.println(line);
            }

            sc.close();
        } catch (Exception e) {
            System.out.println("Error");
        }
    }
}

```

```

import java.io.FileReader;

class Demo {
    public static void main(String[] args) {

        try {
            FileReader fr = new FileReader("test.txt");

            int ch;

            while ((ch = fr.read()) != -1) {
                System.out.print((char) ch);
            }
        }
    }
}

```

```
        fr.close();

    } catch (Exception e) {
        System.out.println("Error");
    }
}
}
```

File Handling Using Byte Stream

Byte stream is used to read and write files **byte by byte**.

The main classes are:

- FileInputStream → **read** bytes from a file
- FileOutputStream → **write** bytes to a file

They are mainly used for **binary files** such as images, audio, video, and also work with text files.

Writing to a File using Byte Stream

```
import java.io.FileOutputStream;

class Demo {
    public static void main(String[] args) {

        try {
            FileOutputStream out = new FileOutputStream("test.txt");

            String s = "Hello Java";

            out.write(s.getBytes());

            out.close();

            System.out.println("Data written");
        }
        catch (Exception e) {
            System.out.println("Error");
        }
    }
}
```

Reading from a File using Byte Stream

```
import java.io.FileInputStream;

class Demo {
    public static void main(String[] args) {
```

```
try {
    FileInputStream in = new FileInputStream("test.txt");
    int ch;
    while ((ch = in.read()) != -1) {
        System.out.print((char) ch);
    }

    in.close();
}
catch (Exception e) {
    System.out.println("Error");
}
}
```

Copy a File Using Byte Stream

```
import java.io.FileInputStream;
import java.io.FileOutputStream;

class Demo {
    public static void main(String[] args) {

        try {
            FileInputStream in =
                new FileInputStream("source.txt");

            FileOutputStream out =
                new FileOutputStream("copy.txt");

            int data;

            while ((data = in.read()) != -1) {
                out.write(data);
            }

            in.close();
            out.close();

            System.out.println("File copied");
        }
        catch (Exception e) {
            System.out.println("Error");
        }
    }
}
```